UNIWERSYTET WSB MERITO WE WROCŁAWIU

PROJEKT KIERUNKOWY

wykonany w ramach studiów podyplomowych

Artificial intelligence. Machine learning

Projekt i weryfikacja prototypu systemu klasy TinyML do rozpoznawania odgłosów zwierząt, bazującego na architekturze STM32.

Autor Projektu:
Bartosz Dykiert
Nr albumu: 53037

Opiekun Projektu:
Dr. inż Kamil Musiał

Wrocław, 2026

Spis treści

1 Wprowadzenie **4**

2 Sieć wykorzystana w projekcie **4**

2.1 Opis sieci neuronowej 4

2.2 Parametry analizy sygnału i ekstrakcja cech 5

2.3 Dane treningowe 6

2.4 Architektura sieci neuronowej 7

2.5 Proces treningowy i callbacki 8

2.6 Konwersja i generowanie plików nagłówkowych dla C 9

3 Aplikacja dekstopowa **10**

3.1 Funkcjonalność interfejsu graficznego 11

4 Oprogramowanie urządzenia **12**

4.1 Główna pętla programu na STM32 15

5 Testy sieci, aplikacji i urządzenia **16**

Spis rysunków

1 Funkcja ekstrakcji cech 6
2 Podział danych treningowych 7
3 Architektura sieci neuronowej 8
4 Proces treningowy - callbacki 9
5 Pliki nagłówkowe dla C 10
6 Ekran główny aplikacji 11
7 Ustawienia zegarow - 1. 13
8 Ustawienia zegarow - 2. 13
9 X cube AI 14
10 Urządzenie wbudowane 14
11 Główna pętla programu na STM32 15
12 Funkcja predykcji z pliku 16
13 Funkcja predykcji z GPT 16
14 Testowanie wszystkich plików za pomocą własnej sieci 17
15 Porównanie predykcji między własną sięcią a modelem GPT 4o audio preview . 17
16 Działanie urządzenia 18
17 Debugowanie urządzenia 18

1 Wprowadzenie

Celem niniejszego projektu dyplomowego było zaprojektowanie i implementacja systemu rozpoznawania dźwięków zwierząt działającego w środowisku systemów wbudowanych. System oparto na mikrokontrolerze STM32 nucle-H753ZI oraz sieci neuronowej stworzonej z użyciem bibliotek tensorflow, numpy oraz sklearn. Projekt składa się z aplikacji na komputer oraz urządzenia wbudowanego. Aplikacja służy do łatwego wizualizowannia danych odbieranych z mikrokontrolera oraz do porównania wyników predykcji z użyciem sieci neuronowej stworzonej na potrzeby projektu oraz z wykorzystaniem platformy open AI w ramach, której został wykorzystany model językowy "gpt 4o audio preview". W ramach pracy zrealizowano akwizycję sygnału audio, ekstrakcję cech MFCC, klasyfikację dźwięków oraz wizualizację wyników na urządzeniu i w aplikacji.

2 Sieć wykorzystana w projekcie

Sieć została stworzona z IDE visual studio code z wykorzystaniem języka Python i bibliotek służących do uczenia maszynowego: tensorflow i sklearn. Dane użyte do nauki sieci pochodzą z publicznego zbioru **ESC-50**. Wybór klasy o najwyższym prawdopodobieństwie odbywa się poprzez operację:

$$\hat{c} = \arg\max_i(y_i) \tag{1}$$

gdzie:

- y_i – wartość wyjścia sieci neuronowej odpowiadająca i-tej klasie,
- $\hat{c}$ – klasa przypisana do analizowanego sygnału.

2.1 Opis sieci neuronowej

W ramach projektu opracowano skrypt w języku Python, wykorzystujący biblioteki TensorFlow/Keras, których zadaniem jest przeprowadzenie pełnego cyklu szkoleniowego sieci neuronowej oraz eksport parametrów do środowiska systemu wbudowanego. Przygotowanie zbioru danych i ekstrakcji cech rozpoczyna się od wczytania surowych plików dźwiękowych w formacie WAV pobranych z zbioru **ESC-50**. Kluczowym etapem jest ekstrakcja cech akustycznych za pomocą funkcji extrac_features. Z każdego nagrania wyznaczane są współczynniki MFCC (ang. Mel-Frequency Cepstral Coefficients). Aby poprawić proces uczenia został zoptymalizowany o wykorzystanie mechanizmu kontrolnego (callback:EarlyStopping). Wszystkie pliki dźwiękowe są w formacie .wav. Następnie została przeprowadzona konwersja do formatu TensorFlow Lite

i wygenerowanie plików nagłówkowych użytych do łatwiejszej implementacji ekstrakcji cech na STM32.

2.2 Parametry analizy sygnału i ekstrakcja cech

Parametry analizy czasowo-częstotliwościowej zostały zdefiniowane w Tabela 1.

Tabela 1: Parametry ekstrakcji cech sygnału audio

Parametr	**Wartość**
Częstotliwość próbkowania (f_s)	44100 Hz
Rozmiar okna FFT (N_{FFT})	1024 próbki
Długość kroku (*hop length*)	512 próbek
Liczba filtrów melowych (N_{mels})	40
Liczba współczynników MFCC (N_{mfcc})	40

Funkcja `extract_features` realizuje przekształcenie surowego sygnału audio w wektor cech o rozmiarze 80. Składa się on z 40 wartości średnich oraz 40 wartości odchylenia standardowego współczynników MFCC obliczonych dla wszystkich ramek czasowych nagrania.

```python
def extract_features(path, n_mfcc=N_MFCC, n_fft=N_FFT, hop_length=HOP_LENGTH, n_mels=N_MELS, sr=SAMPLE_RATE):
    import soundfile as sf
    from scipy.fftpack import dct

    try:
        y, sr = sf.read(path)
        y = y.astype(np.float32)

        # Konwersja stereo -> mono
        if y.ndim == 2:
            y = y.mean(axis=1)

        # Ramkowanie + Hamming
        frames = []
        window = get_window("hamming", n_fft, fftbins=True)
        for start in range(0, len(y) - n_fft + 1, hop_length):
            frame = y[start:start+n_fft] * window
            frames.append(frame)
        frames = np.array(frames)

        # FFT + magnituda
        mag = np.abs(rfft(frames, n=n_fft))

        # Mel filter bank
        mel_fb = librosa.filters.mel(sr=sr, n_fft=n_fft, n_mels=n_mels)
        mel_energy = np.log(np.maximum(mag @ mel_fb.T, 1e-12))

        # DCT -> MFCC
        mfcc = dct(mel_energy, type=2, axis=1, norm='ortho')[:, :n_mfcc]

        # średnia + std
        features = np.hstack([mfcc.mean(axis=0), mfcc.std(axis=0)])
        return features

    except Exception as e:
        print(f"[WARN] Błąd podczas przetwarzania pliku {path}: {e}")
        return None
```

Rysunek 1: Funkcja ekstrakcji cech

2.3 Dane treningowe

Do wyuczenia sieci użyto uczenia etykietowanego. Dane zostały podzielone na:

- Dane trenigowe - 60%
- Dane walidacyjne - 20%
- Dane testowe - 20%

Zastosowano StandardScaler i zapis do formatu .joblib, zapewniając spójność przetwarzania danych w fazie predykcji.

```python
le = LabelEncoder()
y_enc = le.fit_transform(y)

# podzial danych
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y_enc,
    test_size=0.2,
    stratify=y_enc,
    random_state=RANDOM_STATE
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=0.25,
    stratify=y_train_full,
    random_state=RANDOM_STATE
)

#skalowanie
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val   = scaler.transform(X_val)
X_test  = scaler.transform(X_test)

joblib.dump(le, os.path.join(OUTPUT_DIR, "label_encoder.joblib"))
joblib.dump(scaler, os.path.join(OUTPUT_DIR, "scaler.joblib"))
print("[INFO] Zapisano label encoder i scaler.")
```

Rysunek 2: Podział danych treningowych

2.4 Architektura sieci neuronowej

Sieć neuronowa składa się z:

- **Warstwy wejściowej**: rozmiar zgodny z liczbą cech (80).
- **Blok 1**: Warstwa gęsta (256 neuronów, ReLU), Batch Normalization, Dropout (0,3).
- **Blok 2**: Warstwa gęsta (128 neuronów, ReLU), Batch Normalization, Dropout (0,3).
- **Blok 3**: Warstwa gęsta (64 neurony, ReLU), Dropout (0,2).
- **Warstwy wyjściowej**: Warstwa gęsta z funkcją *Softmax*, liczba neuronów odpowiada liczbie rozpoznawanych klas.

```python
num_classes = len(np.unique(y_enc))
input_dim = X_train.shape[1]

model = keras.Sequential([
    keras.layers.Input(shape=(input_dim,)),

    keras.layers.Dense(256, activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(128, activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.2),

    keras.layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Rysunek 3: Architektura sieci neuronowej

2.5 Proces treningowy i callbacki

Model trenowany był z wykorzystaniem optymalizatora **Adam** (learning rate = 10^{-3}) oraz funkcji straty *sparse categorical crossentropy*. W celu optymalizacji procesu wykorzystano trzy mechanizmy kontrolne:

1. **EarlyStopping**: zatrzymanie treningu po 20 epokach braku poprawy błędu walidacyjnego.
2. **ModelCheckpoint**: zapisywanie wyłącznie wag modelu o najlepszych parametrach (*best model*).
3. **ReduceLROnPlateau**: redukcja współczynnika uczenia o mnożnik $0,3$ w przypadku stagnacji procesu minimalizacji funkcji celu.

```python
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=20,
        restore_best_weights=True,
        verbose=1
    ),

    keras.callbacks.ModelCheckpoint(
        filepath=os.path.join(OUTPUT_DIR, "best_model.h5"),
        monitor="val_loss",
        save_best_only=True,
        verbose=1
    ),

    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.3,
        patience=10,
        verbose=1,
        min_lr=1e-6
    )
]

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=args.epochs,
    batch_size=args.batch_size,
    verbose=2,
    callbacks=callbacks
)
```

Rysunek 4: Proces treningowy - callbacki

2.6 Konwersja i generowanie plików nagłówkowych dla C

Finalny model jest eksportowany do formatu `.tflite`.

Kluczowym elementem skryptu jest automatyczne generowanie plików nagłówkowych (`.h`), które przenoszą parametry wyliczone w Pythonie bezpośrednio do kodu C w środowisku STM32CubeIDE:

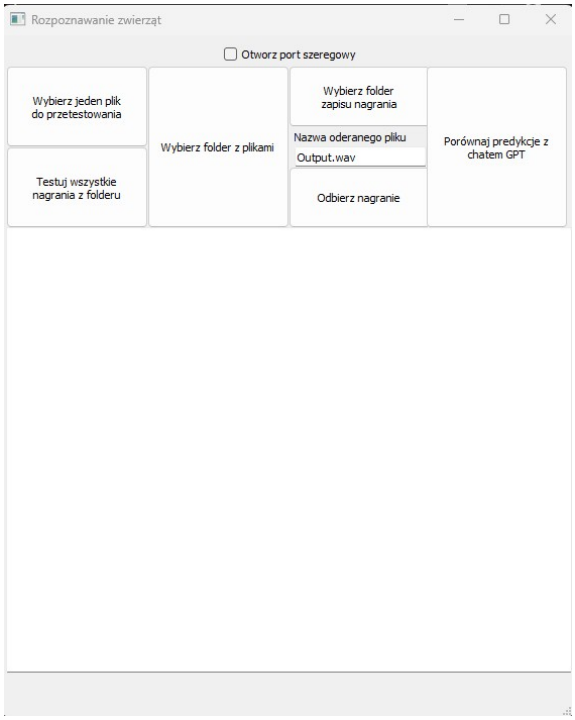
- `scaler_params.h`: zawiera parametry μ oraz σ użyte przez `StandardScaler`.
- `mel_filter_banks.h`: zawiera statyczną macierz filtrów melowych.
- `dct_matrix.h`: zawiera predefiniowaną macierz do obliczeń transformaty kosinusowej.
- `hamming_window.h`: zawiera tablicę wag okna Hamminga.

```python
with open(os.path.join(OUTPUT_DIR, "mel_filter_banks.h"), "w") as f:
    f.write("#pragma once\n\n")
    f.write("// Autogenerated Mel filter banks\n\n")
    f.write(f"#define NUM_MEL_FILTERS {n_mels}\n")
    f.write(f"#define FFT_BINS {n_fft//2+1}\n\n")
    f.write(fmt_matrix("mel_filter_banks", mel_filter))

print("Wygenerowano mel_filter_banks.h")

dct_matrix_manual = np.zeros((n_mfcc, n_mels), dtype=np.float32)
for i in range(n_mfcc):
    dct_matrix_manual[i, :] = np.cos(np.pi * i / n_mels * (np.arange(n_mels) + 0.5))

dct_matrix_manual[0, :] *= 1.0 / np.sqrt(n_mels)
dct_matrix_manual[1:, :] *= np.sqrt(2.0 / n_mels)

with open(os.path.join(OUTPUT_DIR, "dct_matrix.h"), "w") as f:
    f.write("#pragma once\n\n")
    f.write("// Autogenerated DCT matrix\n\n")
    f.write(f"#define NUM_MFCC_COEFFS {n_mfcc}\n")
    f.write(f"#define NUM_MEL_FILTERS {n_mels}\n\n")
    f.write(fmt_matrix("dct_matrix", dct_matrix_manual))

print("Wygenerowano dct_matrix.h")

hamming = np.hamming(FRAME_SIZE).astype(np.float32)

with open(os.path.join(OUTPUT_DIR, "hamming_window.h"), "w") as f:
    f.write("#pragma once\n\n")
    f.write("// Autogenerated Hamming window\n\n")
    f.write(f"#define FRAME_SIZE {FRAME_SIZE}\n\n")
    f.write(fmt_array("hamming_window", hamming))

print("Wygenerowano hamming_window.h")

with open(os.path.join(OUTPUT_DIR, "scaler_params.h"), "w") as f:
    f.write("#pragma once\n\n")
    f.write("// Autogenerated by Python script\n\n")
    f.write(f"#define FEATURE_COUNT {X_train.shape[1]}\n\n")
    f.write(fmt_array("SCALER_MEAN", mean))
    f.write("\n")
    f.write(fmt_array("SCALER_SCALE", scale))
    f.write("\n")
    f.write(fmt_int_array("SCALER_Q14_MEAN", X_train_q14.mean(axis=0).astype(np.int16)))
    f.write("\n")
    f.write(f"static const int SCALER_LEN = {len(mean)};\n\n")
    f.write("static const char *LABELS[] = {\n")
    for lab in labels:
        f.write(f"    \"{lab}\",\n")
    f.write("};\n")
    f.write(f"static const int NUM_LABELS = {len(labels)};\n")

print("Wygenerowano scaler_params.h (Q14 ADC)")
```

Rysunek 5: Pliki nagłówkowe dla C

3 Aplikacja dekstopowa

W celu obsługi systemu oraz wizualizacji wyników opracowano aplikację desktopową z graficznym interfejsem użytkownika (GUI). Interfejs został zaprojektowany w środowisku Qt Designer i zaimplementowany w języku Python przy użyciu biblioteki `PyQt5`. Rysunek **6**. przestawia ekran główny aplikacji.



Rysunek 6: Ekran główny aplikacji

3.1 Funkcjonalność interfejsu graficznego

Główne okno aplikacji zostało stworzone w sposób umożliwiający zarówno testowanie stworzonej sieci na stm32 i wyświetlanie wyników predykcji. Dodatkowym elementem jest możliwość sprawdzenia oceny nagrania wraz z "gpt 4o audio preview". Elementy interfejsu:

- **Otworz port szeregowy** (pole wyboru / checkbox) - służy otwarciu portu szeregowego w celu komunikacji z mikrokontrolerem.
- **Wybierz jeden plik do przetestowania** - wybranie jednego pliku do przetestowania sieci użytej w urządzeniu.
- **Testuj wszystkie nagrania z folderu** - przetestowanie wszystkich plików nagrań.
- **Wybierz folder z plikami** - wybranie folderu z plikami nagrań.
- **Wybierz folder zapisu nagrania** - wybranie folderu, gdzie ma zostać zapisane nagranie z pobranych próbek przez mikrokontroler
- **Nazwa odebranego pliku** (pole tekstowe: Output.wav) - wskazanie nazwy odbieranego pliku.
- **Odbierz nagranie** - klinięcie przycisku, aby powiadomić aplikacje, że zaraz nadejdą dane z urządzenia. Po kliknięciu tego przycisku należy kliknąć przycisk wyślij na mikrokontrolerze.

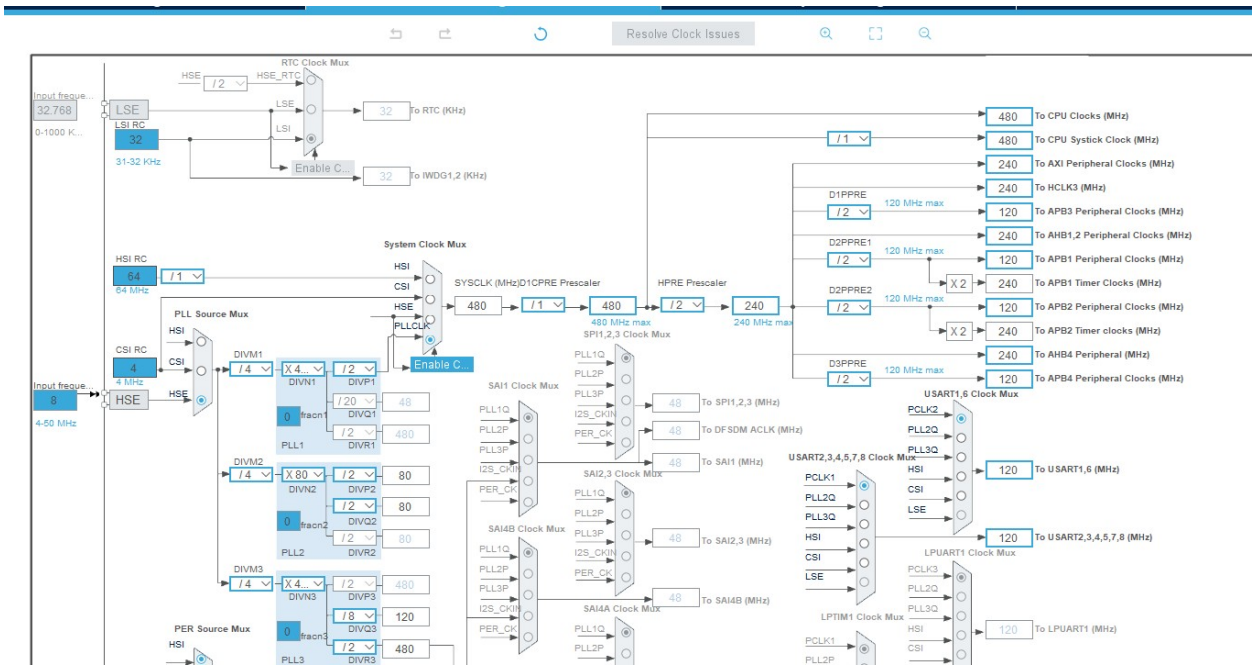
- **Porównaj predykcje z chatem GPT** - wybranie pliku i wysłanie promptu do modelu GPT, aby wyświetlić wynik predykcji od modelu GPT.
- **Pole tekstowe** - służy do wyświetlania komunikatów predykcji oraz komunikatów błędów.

4 Oprogramowanie urządzenia

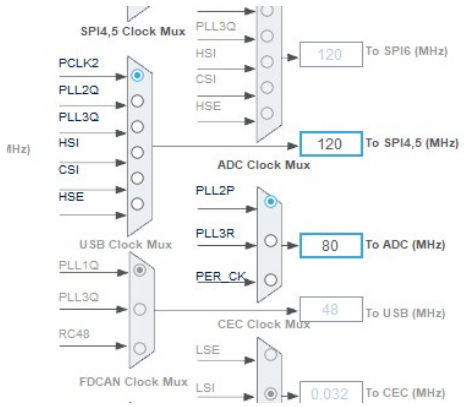
Mikrokontroler został zaprogramowany z użyciem STM32CubeIDE. Przycisk do nagrywania został zaprogramowany na wejściu PA3, odczyt danych z mikrofonu został przeprowadzony przez ADC na wejściu PA4, przycisk do wyświetlenia predykcji na wyświetlaczu SSD1306 został zaprogramowany na wejściu PD1, a przycisk do wysłania nagrania do komputera na wejściu PD0. Do Komikacji z wykorzystaniem interfejsu I2C zostały wykorzystane wejścia PB8 i PB9. Wejścia PB6 i PB15 zostały użyte do komunikacji USART z komputerem. Na Rysunku **7**. oraz Rysunku **8**. są przedstawione ustawienia zegarów. Rysunek **10**. przedstawia wykonane urządzenie. X-CUBE_AI zostało również skonfigurowane do używania wcześniej stworzonej sieci (Rysunek **9**).

Urządzenie składa się z:

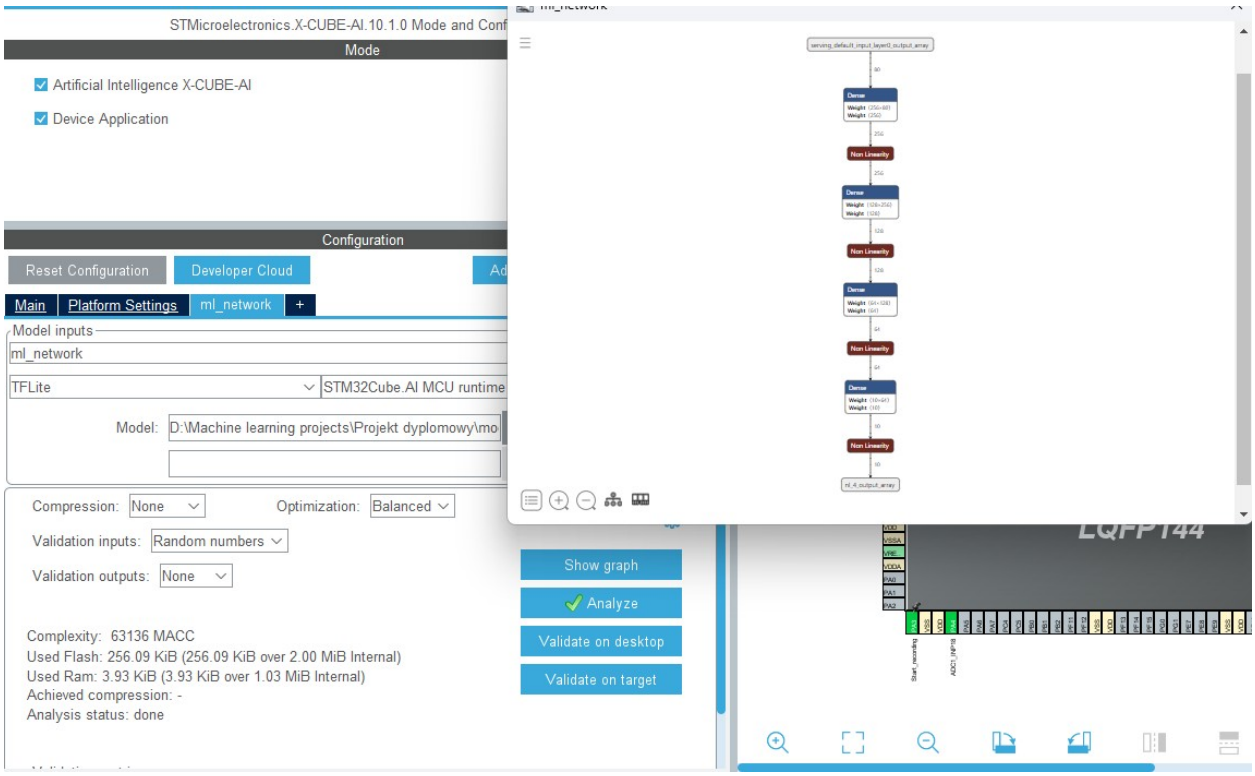
- Trzech przycisków służących kolejno **"1"** do nagrania dźwięku, **"2"** do wysłania nagrania po porcie szeregowym, **"3"**, wyświetlenia predykcji dżwięku.
- Wyświetlacza SSD1306 do wyświetlania komunikatów.
- Mikrofonu elektretowego ze wzmacniaczem MAX9814 do nagrywania dźwięków.
- Pinów RX i TX do komunikacji z komputertem z wykorzystaniem komunikacji USART.



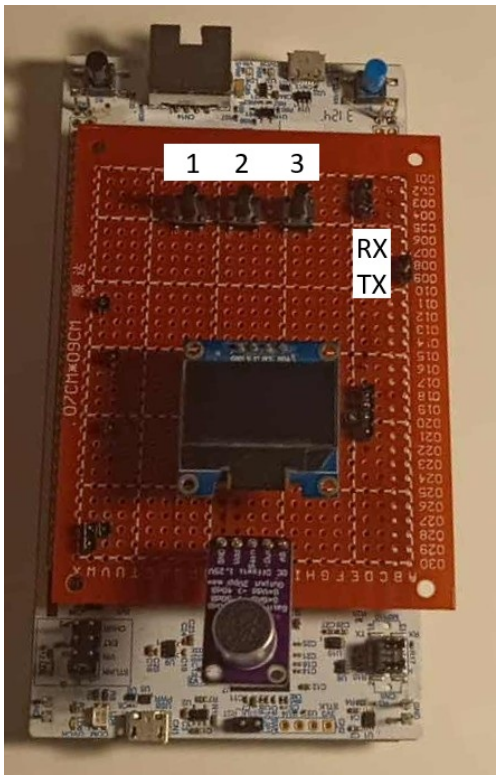
Rysunek 7: Ustawienia zegarow - 1.



Rysunek 8: Ustawienia zegarow - 2.



Rysunek 9: X cube AI



Rysunek 10: Urządzenie wbudowane

4.1 Główna pętla programu na STM32

Główny kod programu na STM32 przedstawia Rysunek **11**. Ze względu na złożoność kodu został przedstawiony tylko ten fragment, gdyż nie jest to główny temat studiów. Mikrokontroler działa niezależnie od aplikacji na komputer. Wgrana jest w niego ta sama sieć, która jest testowana z poziomu aplikacji. Sieć została wgrana za pomocą X Cube AI dostępnego w STM32Cube IDE. Funkcja StartRecording() służy do rozpoczęcia nagrywania.Funkcja run_inference() służy do wprowadzedzenia danych do sieci. Funkcja init_network() służy do zainicjowania sieci wewnątrz mikrokontrolera.

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_ADC1_Init();
MX_USART1_UART_Init();
MX_TIM1_Init();
MX_RAMECC_Init();
MX_I2C1_Init();
MX_X_CUBE_AI_Init();
/* USER CODE BEGIN 2 */

  counter = 0;
  SCB->CPACR |= ((3UL << 10*2)|(3UL << 11*2)); // włącz FPU
  init_network();
  debug_msg("FFT initialization\r\n");
  if (arm_rfft_fast_init_f32(&fft_instance, FRAME_SIZE) != ARM_MATH_SUCCESS) {
      debug_msg("FFT init failed!\r\n");
  }
  if (ssd1306_Init() == 1)
  {
      ssd1306_SetCursor(0,0);
      refreash_display("Initializated");
      HAL_Delay(500);
      debug_msg("SSD1306 success initalizated\r\n");

  }
  else
  {
      debug_msg("SSD1306 wasn't initalizated\r\n");
  }

  app_states = State_idle;

/* USER CODE END 2 */
```

```
switch(app_states)
{
case State_idle:
    debug_msg("IDLE: Press button to start\r\n");
    refreash_display("IDLE");
    break;
case State_recording:
    refreash_display("Recording");
    HAL_Delay(100);
    HAL_TIM_Base_Start(&htim1);   // start timera do delay_us
    debug_msg("Recording");
    StartRecording();
    debug_msg("Recording finished\r\n");
    break;
case State_send:
    refreash_display("Sending");
    HAL_TIM_Base_Stop(&htim1); // Zatrzymaj timer
    SendAllChunks_Blocking();
    send_index =5;
    break;
case State_AI_result:
    refreash_display("AI_result");
    run_inference(audio_buf,NUM_SAMPLES); // wysyłamy wynik predykcji
    app_states = State_idle;      // wracamy do stanu idle
    break;

case State_recording_and_send_chunks:
    refreash_display("send_chunks");
    break;

case State_wait_for_next_step:

    break;
```

Rysunek 11: Główna pętla programu na STM32

5 Testy sieci, aplikacji i urządzenia

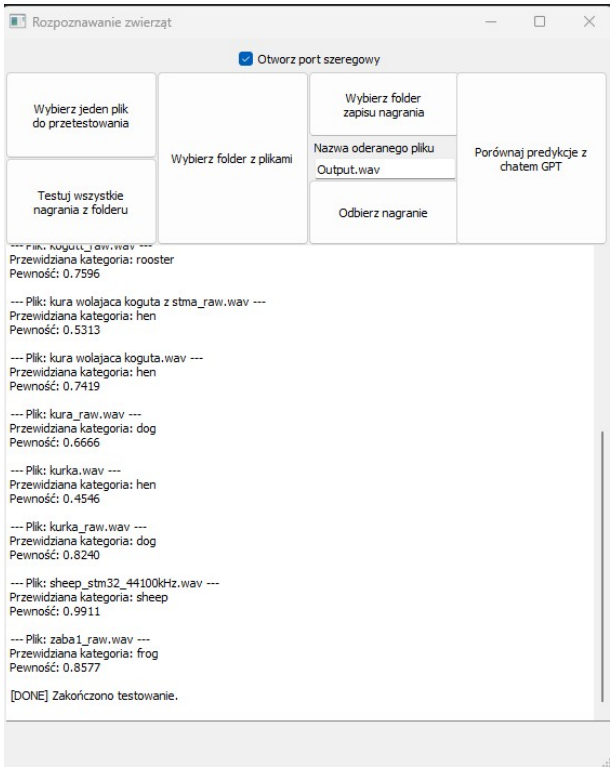
W aplikcje został wbudowany skrypt do testowania wytrenowanej sieci (Rysunek. **12**). Wyniki predykcji sieci potrafią być różne od rzeczywistej, np. koń potrafi być pomylony z kotem (Rysunek. **14**). Po wysłaniu promptu do GPT (Rysunek. **13**). w odpowiedzi otrzymujemy wynik predykcji zgodny z nagraniem (Rysunek. **15**). Najlepiej są rozpoznawane żaby, kury, owce i wrony. Urządzenie zostało przetestowane w następujący sposób, że zostały nagrywane dźwieki zwierząt z głośników z komputera przez urządzenie, po czym na wyświetlaczu SSD1306 można było zobaczyć wynik predykcji wraz z prawdopodobieństwem(Rysunek. **16**). Urządzenie wysyła również komunikaty po porcie szeregowym. Zostało to zrealizowane w ten sposób w celu ułatwienia debugowania podczas pisania kod na mikrokontroler(Rysunek. **17**). Czas trwania każdego nagrania to 5s. Jest ono zapisywane do formatu .wav.

```
def predict_file(self, filepath):

    features = extract_features(filepath)
    if features is None:
        return f"[WARN] Błąd podczas przetwarzania pliku: {filepath}\n"

    # Skalowanie i predykcja
    scaled_features = scaler.transform(features.reshape(1, -1)).astype(np.float32)

    interpreter.set_tensor(input_details[0]['index'], scaled_features)
    interpreter.invoke()
    predictions = interpreter.get_tensor(output_details[0]['index'])

    predicted_class_index = int(np.argmax(predictions))
    confidence = float(predictions[0][predicted_class_index])
    predicted_label = le.inverse_transform([predicted_class_index])[0]

    result = (
        f"--- Plik: {os.path.basename(filepath)} ---\n"
        f"Przewidziana kategoria: {predicted_label}\n"
        f"Pewność: {confidence:.4f}\n"
    )
    return result
```

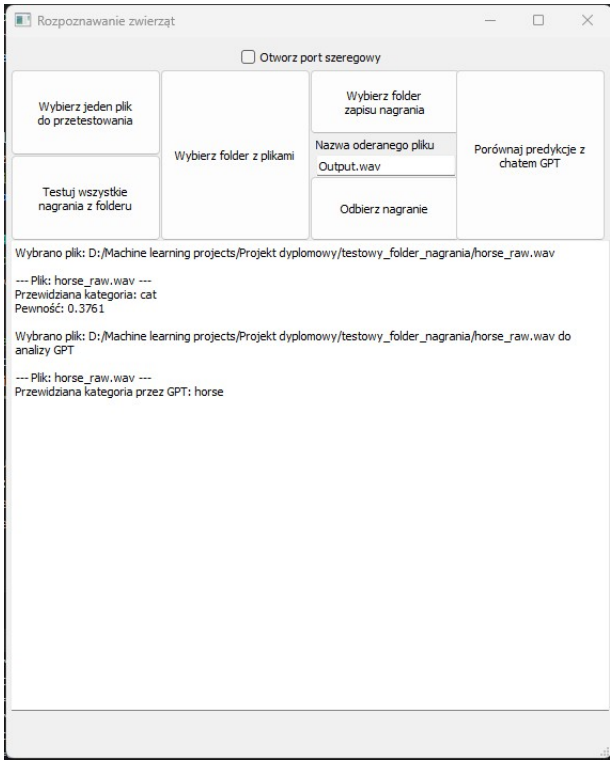
Rysunek 12: Funkcja predykcji z pliku

```
def predict_file_GPT(self, filepath):
    client = OpenAI(api_key=API_KEY_GPT)
    with open(filepath, "rb") as audio_file:
        encoded_audio = base64.b64encode(audio_file.read()).decode("utf-8")
    lista_zwierzat = "cat, cow, crow, dog, frog, hen, insects, pig, rooster, sheep, horse"

    try:

        response = client.chat.completions.create(
            model="gpt-4o-audio-preview",
            modalities=["text"], # Chcemy odpowiedź tekstową
            messages=[
                {
                    "role": "user",
                    "content": [
                        {"type": "text", "text": f"Listen to this audio and tell me which animal from this list it is: {lista_zwierzat}. Return only the name of the animal."},
                        {"type": "input_audio", "input_audio": {"data": encoded_audio, "format": "wav"}}
                    ]
                }
            ]
        )
    except Exception as e:
            return f"ERROR: {e}"
    # 5. Zapisanie odpowiedzi do zmiennej
    predict_result = response.choices[0].message.content.strip().lower()
    result = (
        f"--- Plik: {os.path.basename(filepath)} ---\n"
        f"Przewidziana kategoria przez GPT: {predict_result}\n"
    )
    return result
```

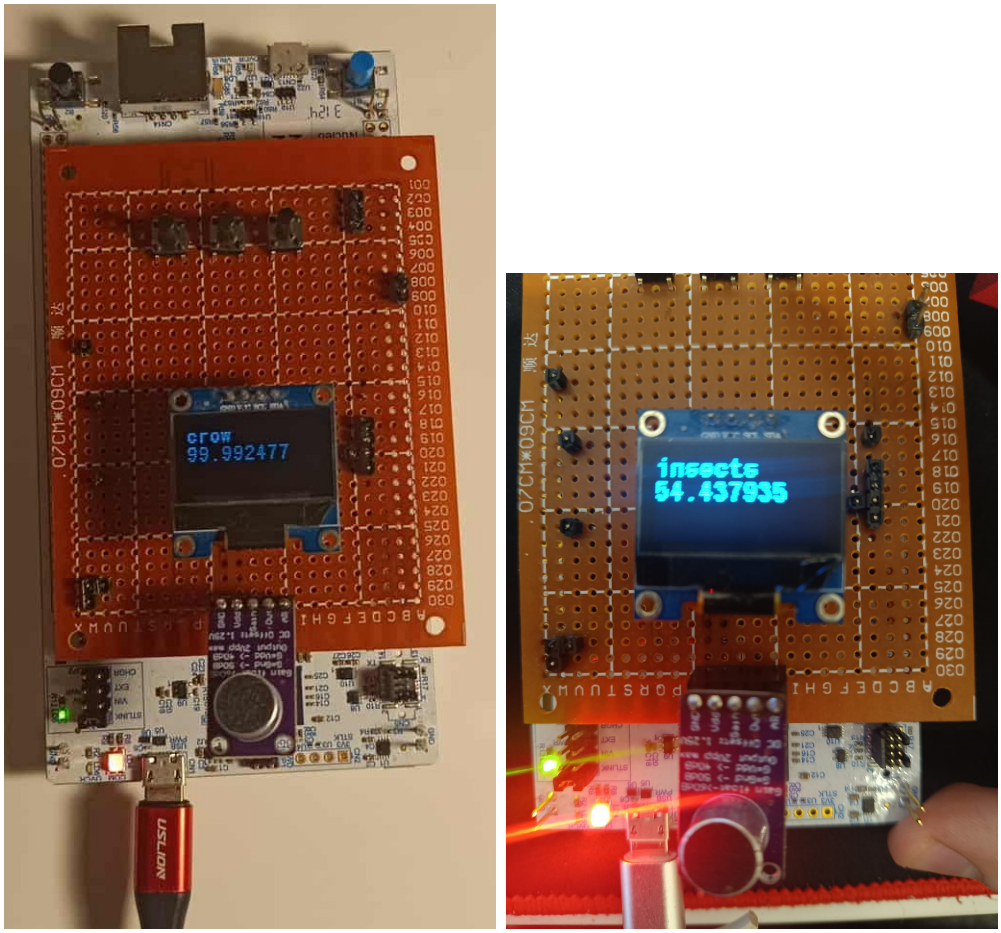
Rysunek 13: Funkcja predykcji z GPT



Rysunek 14: Testowanie wszystkich plików za pomocą własnej sieci



Rysunek 15: Porównanie predykcji między własną sięcią a modelem GPT 4o audio preview



Rysunek 16: Działanie urządzenia

```
compute_mfcc_features DONE
MFCC features - first 10 (before normalization): -25.603018 3.282436 2.253441 1
019317 0.825541 1.255348 1.109038 0.299720 -0.131396 0.248947
MFCC features - first 10 (after normalization): 0.815420 -1.809178 2.130915 0.2
7107 1.729927 0.530378 2.309889 -0.339311 0.296686 -0.390769
Preparing in/out network...
Number of inputs: 1, input number: 1
size output: 80, size: 10
Network running...
Network run succesful
Result of prediction:
Class 0 (cat): 1.56%
Class 1 (cow): 0.18%
Class 2 (crow): 0.49%
Class 3 (dog): 2.50%
Class 4 (frog): 1.46%
Class 5 (hen): 0.13%
Class 6 (insects): 89.51%
Class 7 (pig): 3.06%
Class 8 (rooster): 0.20%
Class 9 (sheep): 0.91%
The highest propability: insects: 89.51%
run_inference finished.
```

Rysunek 17: Debugowanie urządzenia